

SafeNet: Towards mitigating replaceable unsafe Rust code via a recommendation-based approach

Yan Dong | Zhicong Zhang | Mohan Cui | Hui Xu

School of Computer Science, Fudan University, Shanghai, China

Correspondence

Hui Xu, School of Computer Science, Fudan University, 2005 Songhu Road, Shanghai 200438, China.

Email: xuh@fudan.edu.cn

Abstract

Rust is a system-level programming language with advantages in memory safety. It ensures that any Rust programs without unsafe code should not incur undefined behaviours. However, unsafe code still plays an essential role in Rust to achieve low-level control. Therefore, a major design pattern of Rust programs is interior unsafe, which wraps unsafe code as safe APIs and handles all undefined behaviours internally. Rust standard library already provides a rich set of safe APIs to facilitate Rust code development. Nevertheless, due to unfamiliarity with these APIs, developers may misuse unnecessary unsafe code and suffer memory-safety risks. In this paper, we investigate an approach to mitigate replaceable unsafe code. We first analyse unsafe APIs of the Rust standard library and summarize their common usage patterns. Each pattern corresponds to one or several code samples in our knowledge base. Then, we develop an approach to automatically recognize the usage pattern and recommend corresponding code samples. Our approach leverages dataflow analysis to exclude impossible patterns and employs a BERT-based machine learning model to find the most similar pattern among the rest. We have conducted evaluation experiments with 472 unsafe code snippets collected from GitHub projects and successfully recognized the pattern of 394 snippets. We hope our approach can assist developers in detecting unnecessary unsafe code and suggesting safe alternatives.

KEYWORDS

recommendation, Rust, unsafe code

1 | INTRODUCTION

Rust is an emerging system-level programming language that pursues both memory safety and efficiency. However, security and efficiency are two seemingly paradoxical objectives. On one hand, security implies losing some low-level control ability, which may prevent developers from achieving an optimal algorithm in some circumstances. On the other hand, allowing developers to have low-level control would introduce the risk of writing vulnerable code. For example, C/C++ is efficient but vulnerable to dangling pointer issues [1, 2]; Java can be immune to dangling pointer issues because it has no raw pointers and manual memory allocation [3]. To deal with the challenge, Rust separates the language features into an ordinary set (i.e., safe Rust) and a privileged set (i.e., unsafe Rust) [4]. The Rust compiler ensures that any programs written in safe code should not encounter undefined behaviours. If developers need some programs that cannot be achieved with safe code, they can resort to unsafe code by declaring the privilege with an unsafe marker.

Syntactically, Rust defines five types of unsafe code: raw pointer dereference, calling unsafe functions, implementing unsafe traits, mutating static variables and accessing the field of union. These features are useful in various scenarios, such as bypassing the exclusive mutability principle [5] enforced by the Rust compiler via raw pointers and performing type coercion by calling an unsafe function for bit reinterpretation. Nonetheless, developers may inadvertently employ

unnecessary unsafe code that can also be achieved with safe code, which increases the security risks. The steep learning curve of Rust [6] further exacerbates such a problem among Rust beginners, since they may be unfamiliar with Rust APIs, thus referring to the ‘powerful and flexible’ unsafe code [7]. The Rustdoc of unsafe APIs in the Rust standard library has provided dozens of examples of unsafe code that can also be implemented with safe code. For example, some type conversion via the unsafe function `transmute` can also be achieved via `as`.*

In this work, we aim to help developers minimize the unnecessary usage of unsafe code, thereby mitigating the risks of undefined behaviours. To this end, we first study the usage patterns of unsafe APIs in the Rust standard library and whether they can be replaced with safe code. In particular, some usage patterns are misused because their equivalent safe versions incur negligible or trivial overhead. Such misused unsafe code should be avoided in practice because it brings little benefit but only introduces security risks. There is also another type of unsafe code that has safe code substitutions but suffers performance degradation, for example, due to boundary checking. For such unsafe code, we should provide developers with replacement options and let them decide if they can afford the cost.

We model the unsafe code mitigation task as a recommendation problem, that is, given an unsafe code snippet, how to recommend useful proof-of-concept (PoC) code samples as replacement suggestions. We further assume that each unsafe code block should have a major unsafe API call so that we can break down the problem into sub-problems of searching for similar unsafe code usage patterns for specific APIs. Note that our assumption has excluded unsafe traits, static mutable variables and unions, because it is difficult to provide replacement suggestions without considering more project details. Moreover, unsafe API calls and raw pointer dereference are the most prevalent types of unsafe code that occur in nearly 90% and 10% of unsafe blocks, respectively, according to a recent survey [4]. For each unsafe API or raw pointer dereference, we follow the guidance of Rustdoc to enumerate its usage patterns and design the PoCs. Besides, we adjust our classifications based on their real-world usage on GitHub. During the recommendation phase, we first locate the major unsafe API call and then search for similar usage patterns under the same unsafe API.

We mainly analyse the similarity of unsafe code snippets based on the abstract syntax tree (AST) of each unsafe code snippet. Since some patterns require explicit dataflow characteristics related to the unsafe code, we initially perform a coarse-grained dataflow analysis to rule out impossible patterns. To further distinguish the remaining pattern candidates, we serialize the AST and then employ siamese neural networks to learn the embedding of such AST sequences. To elaborate, we first employ Rust code from 41 projects on GitHub to fine-tune BERT so that BERT can learn the representations of general Rust code. Although BERT is a large model, this step is unsupervised learning, and we can easily find a large corpus of unlabelled Rust code for training. However, the model may not be able to learn the details of unsafe code patterns. We thereby add two fully connected layers after BERT and train these new layers with specially labelled unsafe code samples. In particular, the distance of embeddings generated for two code samples of the same pattern should be 0, and 1 vice versa.

We have conducted experiments to evaluate the performance of our approach. Our experiments include 18 most commonly used unsafe APIs according to GitHub statistics, that is, appeared in over 3000 Rust code files. We have systematically summarized their usage patterns and designed PoCs for each pattern. Besides, we have collected dozens of code samples for each unsafe API from GitHub and labelled their patterns. We then perform a k-fold evaluation based on the collected data. Each time, we employ parts of the samples for training and the rest for evaluation. Experimental results show that our approach can achieve a promising performance, with 394 out of 472 correctly recommended.

This work is an extension of our previous conference paper [8] which demonstrated our preliminary results on this problem. In the conference paper [8], we only considered replaceable code pairs but not irreplaceable ones. This work systematically studies the replacement issues for different unsafe APIs and summarises the patterns that could affect the substitutability and replacement strategies. As a result, we have built a new knowledge base that can better solve this problem. Moreover, this work proposes a different code recommendation approach superior to the previous one. To summarise, this paper makes the following contributions:

- It serves as a pilot study to help developers mitigate the incautious usage of unsafe code. We have systematically investigated the problem and categorized unsafe code replacement issues into three types: (1) unsafe code without safe alternatives due to particular features like low-level control; (2) unsafe code that can be directly replaced, for example, replacing unchecked operations with checked; and (3) other unsafe code that might be replaceable depending on its usage patterns.
- We have proposed a hybrid approach for the third type of unsafe code replacement issue. In particular, we model the issue as a recommendation problem. Our approach employs dataflow analysis to rule out impossible patterns and leverages siamese neural networks to learn the embeddings of unsafe code for similarity comparison.

*<https://doc.rust-lang.org/std/mem/fn.transmute.html>.

TABLE 1 Application scenarios and replaceability of the five types of unsafe code in Rustdoc.

Application scenarios	Five types of unsafe code in Rustdoc					Replaceability	
	Raw Ptr	Unsafe Fn	Unsafe Trait	Static Mut	Union	Replaceable?	Alternative Way
Low-level control	•	•	-	-	-	No	-
Interoperability	-	•	-	-	•	No	-
Non-exclusive mutability	•	•	-	-	-	May	Intelligent pointer
Delayed initialization	•	•	-	-	-	May	Initialize twice
Transmute	-	•	-	-	-	May	Safe conversion
Unchecked operations	•	•	-	-	-	Yes	Checked access
Tailored allocator	-	-	•	-	-	No	-
Concurrent objects	-	-	•	-	-	No	-
Global objects	-	-	-	•	-	Yes	Lock

- We have collected a well-labelled dataset containing 472 unsafe code snippets for the 18 most commonly used unsafe APIs. Based on these data, we have conducted evaluation experiments and achieved promising results. We open-source release our dataset and toolkit to facilitate follow-up studies.

2 | UNSAFE RUST AND REPLACEABILITY

In this section, we discuss the usage of unsafe Rust and whether they can be replaced with safe code.

2.1 | Unsafe Rust

Unsafe code plays an important role in Rust because it offers developers fine-grained control over memory. Rust has declared five types of unsafe operations in its language specification,[†] including dereferencing raw pointers, calling unsafe functions or foreign function interfaces (FFIs), accessing mutable static variables, implementing unsafe traits and accessing fields of unions. Due to safety concerns, Rust requires that developers only use unsafe code within the scope annotated with an unsafe marker, which forces developers to be aware of the risks of using the code. Otherwise, the compiler would fail the compilation. In this work, we aim to eliminate incautious usage of unsafe code. However, the current specification of unsafe code types is from the perspective of language syntax and features, and it is very difficult to judge if a particular unsafe code snippet can be replaced or not simply with such information. For each type of unsafe code, there could be cases that can either be replaced or not, depending on the application scenario.

Table 1 demonstrates several typical applications of unsafe code and their associated unsafe code types. Next, we elaborate more on each usage scenario and discuss the necessity.

- *Low-level control*: As a system programming language, it is critical for Rust to provide low-level control capabilities. Take operating system development as an example. We should access the devices (e.g., VGA buffer at address 0xb8000) through raw pointers, and we also need some inline assemblies for I/O. However, both raw pointers and assembly code could incur undefined behaviours. In general, it is difficult to find a safe code to achieve the same goal as that realized by such an unsafe code.
- *Interoperability*: Rust allows interactions with code written in other programming languages, like C/C++. Such interoperability enables developers to directly employ existing libraries or tools and thereby promotes the usability of the language, especially when the Rust community is still young. Since the foreign function interface (FFI) also escapes the safety check of the Rust compiler, it is unsafe and may incur undefined behaviours. Besides, union is a type particularly used for compatibility with C/C++.
- *Non-exclusive mutability*: Exclusive mutability is an important principle enforced by the Rust compiler to achieve memory safety. However, the principle is too rigorous and may limit the language's applicability in some scenarios. Using the data structure of a double-linked list as an example, each node on the list should be accessible with mutability via both its previous node and next node. To address the issue and provide flexible usage, Rust offers a set of

[†]<https://doc.rust-lang.org/book/ch19-01-unsafe-rust.html>.

unsafe APIs that construct objects from raw pointers in its standard library, for example, the `from_raw_parts()` method of `Vec`. Note that the novel exclusive mutability checking performed by the Rust compiler is one of the key reasons for the steep learning curve. Therefore, novice developers may prefer using unsafe code simply because such code is compilable, even though it is unnecessary. Such problems are our paper interests. Besides, there are other alternative safe solutions in Rust that resort to runtime checks, like shared references (i.e., `Rc`) or internal mutability (e.g., `Cell` or `RefCell`). However, these solutions incur execution overhead and may not be acceptable for system development.

- *Delayed initialization*: Rust pursues RAII (resource acquisition is initialization), that is, each object should be well initialized when being created. RAII is a prerequisite for Rust to safely drop objects in an automatic and recursive way. For example, if a composite type contains other composite types of `Drop` trait as its fields, the program may end up dropping uninitialized memory if the inner field is not initialized. It is possible to write such buggy code via unsafe code, for example, with the `uninitialized()` function or `assume_init()` of the `MaybeUninit` type. Unsafe Rust offers delayed initialization because the mechanism is useful in some scenarios, for example, to avoid the memory being written twice and improve the performance. Nonetheless, similar to non-exclusive mutability, RAII poses constraints to developers when coding. As a result, novice developers may use unnecessary unsafe code and incur security risks.
- *Transmute*: Rust is a type-safe language, which means it does not implicitly convert the value of one type to another. Developers should use `as` to safely perform conversions among primitive types or implement their own conversion methods for other customized types. On the contrary, `transmute` is an unsafe bitwise casting function that arbitrarily converts the memory of one type to another without considering the actual memory content, for example, directly using the bits of a float as `i32`. It is unsafe because it requires developers to guarantee the consistency of the type conversion.
- *Unchecked operations*: For security reasons, safe Rust performs a boundary check by default when accessing the items of a container. Unsafe Rust also offers unchecked APIs that remove boundary checks to avoid its runtime overhead. Unchecked APIs may lead to out-of-bound access and incur memory-safety issues. Besides, unsafe Rust offers several other unchecked operations. For example, arithmetic overflow is an undefined integer in Rust, developers can use `unchecked_add` to bypass the overflow check; converting raw bytes to strings in an unchecked mode may incur undefined behaviours if the bytes are invalid utf-8.
- *Tailored allocator*: Although Rust provides a default global allocator, it also allows developers to customize their own allocator to better fit their needs. In particular, developers can implement the unsafe trait `GlobalAlloc` for their allocator and set it as the default allocator with the `pragma #[global_allocator]`. Since the allocator manages the allocated heap units, falsely implementing the allocation and deallocation functions would incur memory-safety problems.
- *Concurrent objects*: Sharing memory units among multiple threads makes a program vulnerable to race condition that it may incur unpredictable inconsistencies during execution. To prevent such risks, Rust enforces that all objects should not be shared among threads by default. Besides, it provides unsafe traits `Send` and `Sync` so that developers can share data by manually implementing the traits for their data structures. However, it relies on the developers to ensure the correctness of concurrency (e.g., atomicity or synchronization) or thread safety. Falsely implementing these traits would lead to concurrency bugs.
- *Global objects*: Global objects live throughout the whole life span of the program. Mutating a global object should be unsafe because there can be other mutable or immutable references to the object at the same program point and in an inter-procedural way. For example, if a function A calls function B, and function B modifies a global object, it would be difficult to check whether other references to the object hold when calling B via intra-procedural analysis. One alternative safe way of using global objects is to wrap them within a lock, for example, a `Mutex` lock.

2.2 | Replaceable unsafe code

In this paper, we define replaceable unsafe code as a snippet of unsafe code that can be implemented without directly using unsafe code. Note that the corresponding safe version may be based on other library APIs with interior unsafe code [9]. The assumption is valid because interior unsafe is a major underlying principle of Rust to avoid undefined behaviours by preventing developers from using unsafe code directly.

Next, we demonstrate two typical cases of unsafe code replacement: *misused unsafe code* and *replacement with additional cost*.

2.2.1 | Misused unsafe code

Misused unsafe code means the code can be replaced with an equivalent safe code at a trivial cost. Figure 1 demonstrates a real-world example based on a GitHub project change history. We have simplified the original code for better visualization. The example contains a `rs_template_state_new` method (line 1) that returns a raw pointer of `c_void` type. However, the initial value obtained from `Box::new()` is `Box<TemplateState>` type (line 3). Since Rust is type-safe, developers should perform type coercion explicitly before returning the value. In the old version of Figure 1a, the developer uses an unsafe function `transmute` to perform type conversion. Nevertheless, as shown in Figure 1b, this can also be achieved via a safe function `as`. In this way, the unsafe code is unnecessary. We think a major reason for such misused unsafe code is due to unfamiliarity with Rust. There is another situation where only the unsafe marker is misused, that is, there is no unsafe code within the scope annotated by an unsafe marker. We do not consider such issues as misused unsafe code because the code does not perform privileged behaviours.

2.2.2 | Replacement with additional cost

Besides misused unsafe code, developers may intentionally employ unsafe code for efficiency. We now use a program of double-linked list in Figure 2 to demonstrate the issue. A double-linked list requires that each intermediate node should be mutably accessible through its previous and next node. The requirement obviously conflicts with the mutual exclusive principle of Rust. To bypass the compiler check, using unsafe code is a straightforward way. As shown in Figure 2a, we can define the previous and next fields (lines 3–4) of a node and the head and tail fields (lines 7–8) of the list as raw pointers. To push a new element into the front of the list, we need to dereference the raw pointers of the head (line 23) to access the previous node pointer and link it to the new element. The program can also be implemented with safe code only through intelligent pointers. Figure 2b demonstrates the safe version, and we can define the previous and next fields as `Option<Rc<RefCell<Node>>>` (lines 4–5). `Rc` is a type of intelligent pointer in Rust, and `RefCell` is used for interior mutability. Both of the two constructs employ runtime checking and introduce non-trivial overhead.

3 | PROBLEM DEFINITION AND ANALYSIS

In this work, we aim to detect replaceable unsafe code from Rust projects and recommend the safe version to developers. Developers can decide whether to use the recommended version themselves by considering the overhead issues or other side effects. Since understanding and rewriting a code snippet is generally hard due to Rice's Theorem [10], the purpose of this paper is to recommend similar unsafe code patterns and their corresponding safe versions to developers.

3.1 | Problem definition

Suppose we have a dataset of well-prepared unsafe code (i.e., knowledge base) of different patterns and their safe alternatives if exist, denoted as $D = \{(u_i, s_i) \mid i \in [0, n)\}$. Given a code snippet within an unsafe marker, our problem is how to find the most similar case $d_j \in D$ and recommend it to developers. One critical issue of the problem is how the knowledge base is prepared. To better solve the problem, we may further assume that each unsafe code snippet corresponds to one unsafe API call, $D = \{(u_i, s_i) \in API_j \mid i \in [0, n), j \in [0, m)\}$. The assumption is valid because the number of unsafe APIs in the Rust standard library is limited. If there are multiple unsafe API calls within one unsafe code region, we can categorize them based on the most primary API. Our approach may still fail if a real-world unsafe code snippet is

```
1 fn rs_template_state_new () -> *mut std::os::raw::c_void {
2     let state = TemplateState::new();
3     let boxed = Box::new(state);
4     return unsafe { transmute(boxed) };
5 }
```

(a) Unsafe version that performs type coercion via `transmute`.

```
1 fn rs_template_state_new () -> *mut std::os::raw::c_void {
2     let state = TemplateState::new();
3     let boxed = Box::new(state);
4     return Box::into_raw(boxed) as *mut c_void;
5 }
```

(b) Safe version that performs type coercion via `as`.

FIGURE 1 A simplified example from GitHub (project: OISF/suricata) that misuses unsafe code during type coercion.


```

1 struct Node {
2     val: i32,
3     next: *mut Node,
4     prev: *mut Node,
5 }
6 struct List {
7     head: *mut Node,
8     tail: *mut Node,
9 }
10
11 impl List {
12     pub fn new() -> Self {
13         List {head: ptr::null_mut(), tail: ptr::null_mut()}
14     }
15     pub fn push(&mut self, value: i32) {
16         let new = Box::new(
17             Node { val: value,
18                   prev: ptr::null_mut(),
19                   next: ptr::self.head});
20         let raw = Box::into_raw(new);
21         if !self.head.is_null() {
22             unsafe {
23                 (*self.head).prev = raw;
24             }
25         } else {
26             self.tail = raw;
27         }
28         self.head = raw;
29     }
30 }

```

(a) Double-linked list with unsafe code.

```

1 #[derive(Clone)]
2 struct Node {
3     val: i32,
4     prev: Option<Rc<RefCell<Node>>>,
5     next: Option<Rc<RefCell<Node>>>,
6 }
7 struct List {
8     head: Option<Rc<RefCell<Node>>>,
9     tail: Option<Rc<RefCell<Node>>>,
10 }
11 impl List {
12     pub fn new() -> Self {
13         List {head: None, tail: None}
14     }
15     pub fn push(&mut self, value: i32) {
16         let n = Node {val: value, prev: None, next: None};
17         let new = Rc::new(RefCell::new(n));
18         match self.head.take() {
19             Some(h) => {
20                 h.borrow_mut().prev = Some(new.clone());
21                 new.borrow_mut().next = Some(h);
22                 self.head = Some(new);
23             }
24             None => {
25                 self.head = Some(new.clone());
26                 self.tail = Some(new);
27             }
28         }
29     }
30 }

```

(b) Double-linked list without unsafe code.

FIGURE 2 Two ways to implement a double-linked list.

large and contains several unsafe API calls. However, such a large unsafe code region should not be encouraged in practice because we should narrow down the unsafe code region to facilitate code review or security checking. Next, we discuss our detailed knowledge base preparation method.

3.2 | Knowledge base construction

To construct a database with useful code pairs, we first resort to the Rustdoc standard library as the main source. Rustdoc provides code samples to instruct developers on how to use each API, and it also contains alternative safe ways of using unsafe APIs. For example, the code pairs of unsafe API `mem::transmute` in Figure 3a–b, Figure 3c–d and Figure 3e–f are directly collected from Rustdoc.[‡] Our detailed construction approach contains two steps: (1) *enumerating unsafe APIs in std-lib* and (2) *designing code samples for each API*.

3.2.1 | Overview

We manually search unsafe APIs with the keywords ‘pub unsafe fn’ from the Rustdoc of the Rust standard library. There are two types of unsafe APIs: static APIs and dynamic APIs. A static API means the API can be called directly via its namespace and method name, for example, `mem::transmute`; a dynamic API means the API should be called

[‡]<https://doc.rust-lang.org/std/mem/fn.transmute.html>, accessed in June 2023.

```
1 let p = &Q;
2 let p = unsafe { mem::transmute:: <&i32, usize >(p) };
```

(a) Case 1 (pattern 1) with misused unsafe code transmute .

```
1 let raw_bytes = [0x78, 0x56, 0x34, 0x12];
2 let num = unsafe {
3     mem::transmute:: <[u8; 4], u32>(raw_bytes) };
4 assert_eq! (num, 0x12345678);
```

(c) Case 2 (pattern 2) with misused unsafe code transmute .

```
1 let store = [0, 1, 2, 3];
2 let v = store.iter().collect:: <Vec<&i32 >>();
3 let v = unsafe {
4     mem::transmute:: <Vec<&i32 >, Vec<Option <&i32 >>>(v)
5 };
```

(e) Case 3 (pattern 3) with unsafe code transmute for direct type conversion.

```
1 struct Foo<T> { v: T }
2 impl <T> Foo<T> {
3     pub fn getptr (x: &mut Box<Self >) -> *mut libc::c_void {
4         let ptr: *mut Self = x.as_mut();
5         unsafe { mem::transmute(ptr) }
6     }
7 }
```

(g) Case 4 (pattern 1) that can also be replaced with as .

```
1 let ptr = &Q;
2 let ptr = ptr as *const i32 as usize ;
```

(b) Safe version of Case 1 that uses as instead.

```
1 let raw_bytes = [0x78, 0x56, 0x34, 0x12];
2 let num = u32::from_le_bytes(raw_bytes);
3 assert_eq! (num, 0x12345678);
4
```

(d) Safe version of Case 2 that uses u32::from_le_bytes()

```
1 let store = [0, 1, 2, 3];
2 let v = store.iter().collect:: <Vec<&i32 >>();
3 let v = v.into_iter().map( Some )
4     .collect:: <Vec<Option <&i32 >>>();
5
```

(f) Safe version of Case 3 that incurs additional execution overhead.

```
1 let p = &mut 0;
2
3 let val_transmuted = unsafe {
4     mem::transmute:: <&mut i32, &mut u32>(p)
5 };
6
7
```

(h) Case 5 (pattern 1) that cannot be replaced with as .

FIGURE 3 Demonstrating our knowledge base construction approach and the challenge of code recommendation problem with an unsafe API `mem::transmute()`.

TABLE 2 Unsafe APIs in the Rust standard library.

Application scenario	Sample unsafe APIs
Non-exclusive mutability	<code>Box::from_raw</code> , <code>Rc::from_raw</code> , <code>Vec::from_raw_parts</code> , <code>String::from_raw_parts</code> , ...
Delayed initialization	<code>assume_init</code> , <code>assume_init_ref</code> , <code>assume_init_read</code> , <code>mem_zeroed</code> , <code>array_assume_init</code> , <code>set_len</code> , ...
Transmute	<code>mem::transmute</code> , <code>mem::transmute_copy</code> , <code>align_to</code> , <code>align_to_mut</code> , ...
Unchecked operation	<code>Arc::get_mut_unchecked</code> , <code>get_unchecked</code> , <code>new_unchecked</code> , <code>offset</code> , <code>String::from_utf8_unchecked</code> , ...
Others	<code>*</code> , <code>ptr::write</code> , <code>ptr::read</code> , <code>ptr::drop_in_place</code> , <code>alloc::alloc</code> , <code>add</code> , ...

via its instantiated object, and there could be various implementations of the API for different types of objects, for example, `v.assume_init`. Furthermore, we also consider raw pointer to be an intrinsically unsafe API.

Table 2 presents several sample unsafe APIs for each typical application scenario. For example, `Box::from_raw` is a static API that constructs an object of the target type based on a raw pointer as the input. Since the API turns a piece of memory into an owned object, it may incur multiple owners of the memory unit and cause memory-safety issues. `assume_init` is a dynamic function that converts a `MaybeUninit<T>` object to an initialized one. The API is

implemented in several data structures, such as Box, Arc and MaybeUninit. It is difficult to locate the particular function without inferring the object type.

Overall, these unsafe APIs can be categorized into three types based on whether and how they can be replaced.

- *Straightforward replacement*: Such unsafe APIs can be directly replaced with specific safe APIs. All APIs for unchecked operations are of this type. The replacement strategy is mainly to remove the string ‘unchecked’ from the API identifier and obtain the safe version. For example, the safe version of `Arc::get_mut_unchecked` is `Arc::get_mut`. Besides, several raw pointer APIs are unsafe by default, and they can be directly replaced with the wrapping versions, which are safe. For instance, `add` is a dynamic function implemented for the `ptr` type for calculating a new address. It is unsafe because the resulting address may exceed the maximum. `wrapping_add` is the corresponding safe version that performs the limit check.
- *Irreplaceable*: There are also some unsafe APIs that do not have safe alternatives due to their unique low-level controls. For example, `ptr::read_volatile` and `ptr::write_volatile` are two of several limited APIs with volatile memory access features. To our best knowledge, we cannot find other safe replacement APIs that enforce volatility. Another example is the reference count manipulation operations for intelligent pointers, such as `Rc::decrement_strong_count` and `Rc::increment_strong_count`.
- *Maybe replaceable*: Such type accounts for most of the unsafe APIs. They could be replaceable in certain contexts with specific approaches, but they cannot be replaced in other contexts. Take `mem::transmute` as an example, all the cases in Figure 3a, c, e, g can be replaced, and they could have diversified replacement ways. However, the case in Figure 3h cannot be easily replaced. Such categories of unsafe APIs are the most complicated ones, and we will especially focus on such APIs in the following study.

3.2.2 | Design code patterns

For each unsafe API, there could be enormous usage cases. However, we can summarize their usage patterns and let each pattern correspond to one or several limited-safe substitution strategies. In this work, we consider the function scope of each unsafe API usage as a case. When summarizing code patterns, we consider both the structure of the unsafe code and its replacement methods. If the structures of two unsafe functions share some similar patterns that may determine the replacement strategy, we think they belong to one pattern. Table 3 lists our summarized patterns for each popular unsafe API. We now employ `mem::transmute` to discuss our detailed pattern design methods. One main factor that affects the replacement strategies of the API is the type of parameter. Pattern 1 represents the conversions between scalar types that might be replaceable with the safe API `as`. For example, Figure 3a is a case of Pattern 1 that converts a value of `&i32` to `usize`. The conversion should be safe because the size of `usize` corresponds to the addressing mode (e.g., 32 or 64 bit) of the target machine, and `usize` can properly represent any memory address of the machine. Therefore, Rustdoc suggests developers use `as` instead, which is safe. Figure 3c is a different pattern that converts a slice of raw bytes to `u32`. The functionality can also be achieved via a safe function `u32::from_le_bytes`. We do not categorize it as Pattern 1 because it cannot be replaced with `as`. Figure 3e is another different pattern that converts an object of `Vec<i32>` to `Vec<Option<i32>>`, and its replacement method differs from all previous patterns. That is, the safe alternative iterates over each item of the object to perform type coercion. Besides, there are three other patterns that cannot be replaced. Pattern 4 converts a raw pointer to a reference. There is no safe method that can achieve the goal because the operation may incur dual owners or violate the exclusive mutability principle. Pattern 5 extends or shortens the lifetime of references. Pattern 6 converts a `MaybeUninit` object to an ordinary object. Such conversion can also be achieved via `assume_init`, which is another unsafe API.

The main factors that affect the replacement strategies could be diversified for different APIs. For example, the unsafe constructors (e.g., `Rc::from_raw` and `Vec::from_raw_parts`) create an owned object based on a raw pointer parameter. Therefore, the essential reason that affects its replaceability should be where the raw pointer comes from. If it comes from the function entry as a parameter value (Pattern 1) or is returned by a function call (Pattern 3), it would be difficult for us to replace the unsafe code without modifying other functions or refactoring the project. Besides, there are four patterns that might be replaceable. Pattern 2 accepts a reference (e.g., `&self`) as input, which is later employed as the parameter of the unsafe constructor. The replaceability of such cases depends on the field type of the reference being used, for example, irreplaceable if it is a raw pointer. Besides, there are three replaceable patterns because all the contents pointed to by the raw pointer parameter are created within the current function. We currently do not discriminate between replaceable and irreplaceable ones of similar structures as different patterns. We believe our approach can already provide useful suggestions to developers so that they can easily try the suggestions if they use

TABLE 3 Typical patterns of several widely used unsafe APIs (# of GitHub records > 3000).

API	Patterns		Replacement	
	Pattern description	Filter	Replaceable?	Replacement strategy
Unsafe constructors: Box::from_raw String::from_raw_parts Vec::from_raw_parts ...	1: the caller accepts a raw ptr parameter used by the constructor	Yes	No	-
	2: the caller accepts a ref parameter used by the constructor	Yes	May	there could be safe methods
	3: the API employs a raw ptr returned by other static functions calls	Yes	No	-
	4: get the object ptr for other APIs and then reconstruct it	Yes	Yes	into_raw => as_ptr
	5: construct an object based on another object of a different type	No	Yes	there should be safe methods
	6: modify the content of the object	No	Yes	there should be safe methods
mem::transmute	1: type conversion between scalar types	Yes	May	use as
	2: type conversion involves primitive compound types	Yes	May	use specific safe APIs
	3: type conversion between compound types with iterators	Yes	Yes	iterate and convert
	4: convert raw to reference	Yes	No	-
	5: extend or shorten the lifetime	Yes	No	-
	6: from MaybeUninit to initialized objects	Yes	No	-
mem::zeroed	1: create an object	No	May	init with Default::default()
	2: reinitialize the object through another function via raw pointers	Yes	May	Default::default() or MaybeUninit
assume_init	1: create an uninit object and then init	No	Yes	init and rewrite later
	2: the caller accepts a MaybeUninit parameter and initializes it	Yes	No	-
	3: get a return value of MaybeUninit and initializes it	Yes	No	-
	4: initialize the object in another (especially FFI) function	Yes	No	-
ptr related: ptr::{read, write, ...}	1: the caller accepts a raw ptr parameter used by the unsafe API	Yes	No	-
	2: the caller accepts a ref parameter used by the unsafe API	Yes	Yes	-
	3: the API employs a raw ptr returned by other static function calls	Yes	No	-
	4: the API employs a raw ptr obtained from an owned object	Yes	May	-
set_len	1: reserve space, init via another function, and then set len	No	May	new, and then pop or drain
	2: reserve space, set len, and then init via another function	No	May	pattern 1 is safer
	3: vector shrink: to avoid dual ownership	No	No	-

Note: Filter: the pattern has necessary dataflow (as well as types) conditions that should be satisfied in our study.

unsafe APIs due to incautious reasons. Figure 3h provides an example that converts an i32 reference to a u32 reference. We categorize it as Pattern 1 because both i32 and u32 are scalar types. However, the conversion of their references is undefined, and it cannot be achieved via as. If we can recommend the code pairs of Figure 3a and Figure 3b to developers, they can easily try as and know it is irreplaceable. Another critical issue that matters to the recommendation performance is the richness of cases in each pattern. For example, the real-world case in Figure 1a should have the same pattern as Figure 3a because their unsafe code transmute can both be replaced with as.

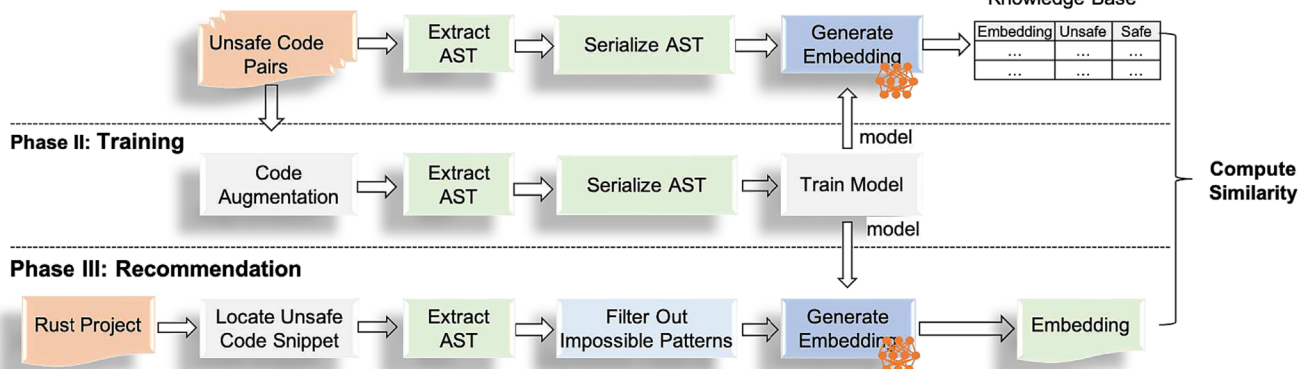
Phase I: Preparing Knowledge Base

FIGURE 4 Overall framework of our code recommendation approach.

However, they do not look very similar. To solve the problem and benefit the following recommendation task, our solution is to add more representative and self-contained cases for each pattern, for example, adding a new case of Figure 3g, representing converting a pointer or reference of a boxed data to `c_void`. Although each unsafe code pattern may exhibit enormous variations and enumerating all of them is difficult at one time, if not impossible, we can still enrich the knowledge base gradually in the long run. As long as new representative cases are found in the future, we will add them to the knowledge base. Besides, with more cases found, we may need to further refactor the classification of patterns, for example, dividing one pattern into several sub-patterns or creating a new pattern. We leave the enrichment of the knowledge base as our future work. In this paper, we mainly focus on how to build such a knowledge base and the approach to making recommendations.

4 | PATTERN RECOGNITION APPROACH

As shown in Figure 4, our approach includes three phases: a *knowledge base preparation phase* that collects unsafe code samples and generates embedding for each sample; a *training phase* that builds a machine learning model to learn the embedding of code snippets; and a *recommendation phase* that applies the machine learning model to find the most similar unsafe code from the knowledge base. Among these phases, there are three critical steps that affect the accuracy of machine learning, which are AST extraction and serialization, dataflow-based pattern filter, and model training. Next, we elaborate our detailed method for each step.

4.1 | AST extraction and serialization

In order to apply machine learning algorithms to discriminate unsafe code snippets, we should extract a representation of these code snippets. There can be different choices of representations, such as using source code, abstract syntax tree (AST), attributed control-flow graph (ACFG) and linear intermediate representation (IR). In this work, we prefer AST-based approaches because AST is well structured and has retained all the code information. In comparison, ACFG only contains information about predefined features for code blocks; linear IR might be too low-level for this problem. Rust AST is also named HIR (high-level intermediate representation). We can easily extract the HIR of a Rust code snippet with the Rust compiler and the option `-emit-hir` or with the `syn` library [11]. The original HIR is very complicated because the Rust compiler introduces many assistant nodes and attributes during parsing. However, we only want to leverage the power of AST to better capture the relationship between tokens in a Rust code snippet. Therefore, we simplify the original HIR into a most simplified representation that meets the following standard:

- Each leaf node should be a token node.
- The detailed information of all intermediate nodes can be discarded.
- Each intermediate node should have multiple branches; otherwise, it can be removed from the AST. Algorithm 1 demonstrates our pruning method.

Algorithm 1 Pruning redundant intermediate node for Rust AST

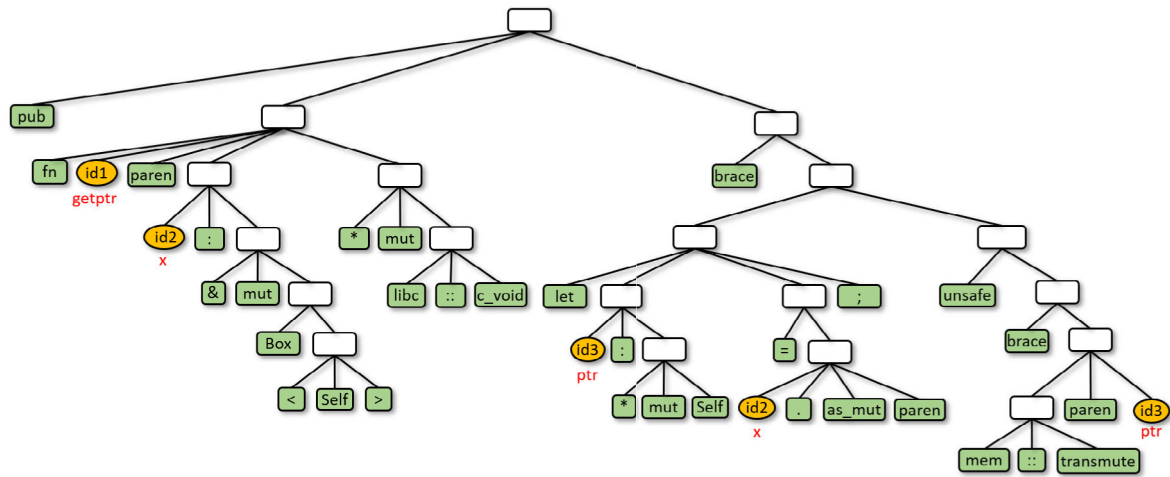
```

1: procedure PRUNE(root)
2:   node  $\leftarrow$  root;
3:   stack  $\leftarrow$  {};
4:   while node  $\leftarrow$  DFSVisit(node) do
5:     if !node.isLeafNode() then
6:       stack.push(node);
7:       continue;
8:     else
9:       while p  $\leftarrow$  stack.pop() && p  $\neq$  NULL do
10:        if p.childNum() > 1 then
11:          stack.push(p);
12:          break;
13:        else
14:          RemoveFromAST(p);
15:        end if
16:      end while
17:    end if
18:  end while
19: end procedure

```

▷ Create an empty stack to save nodes from the root to the current node.
 ▷ Visit each node of the AST following the DFS order.
 ▷ Intermediate node: push it onto the stack and continue.
 ▷ Leaf node: remove its ancestor nodes that have only one branch.
 ▷ Stop if it has more than one branches.
 ▷ The node has only one branch.

We then perform identifier simplification on the AST. In particular, we replace the lexical information of developer-named identifiers with placeholders while retaining the identifiers of types, domains, function calls and macros from the standard library. Figure 5a



(a) Simplified abstract syntax tree.

```

3, pub,
5, fn, id1, paren,
  3, id2, :, 3, &, mut, 2, Box, 3, <, Self, >, N,
  3, *, mut, 3, libc, ::, c_void,
2, brace,
  2, 4, let, 3, id3, :, 3, *, mut, Self, 2, =, 4, id2, ., as_mut, paren, ,,
  2, unsafe, 2, brace, 3, 3, mem, ::, transmute, paren, id3

```

(b) Preorder traversal sequence with extra information recording the branch number for each intermediate node.

FIGURE 5 Example of code preprocessing.

demonstrates an example of the simplified AST for the unsafe code of Figure 3g. Note that we have replaced the two occurrences of 'ptr' with 'id3'. The replacement is critical for discriminating between the domain of 'ptr' and a variable name. Next, we serialize the AST nodes into a sequence of tokens via preorder traversal. Since there could be multiple trees that share the same preorder traversal sequence, we record the number of branches for each intermediate node as assistant information to ensure its uniqueness. For example, Figure 5b shows the serialization of the AST in Figure 5a.

4.2 | Dataflow-based pattern filter

According to the pattern descriptions of each API in Table 3, it is obvious that some patterns should satisfy particular conditions related to the dataflow and types. For example, the first pattern of `Box::from_raw(p: *mut T)` implies there is a backward dataflow from the parameter `p` to a raw pointer parameter of the caller function, while the second pattern implies a dataflow to a reference parameter of the caller function; the patterns of `mem::transmute()` require checking the type of two parameters. Therefore, we can extract and analyse the dataflow as well as the types to perform an initial classification. However, the power of such analysis is limited because a code snippet may demonstrate the characteristics of multiple patterns, and there are also some patterns involving semantic analysis, for example, the sixth pattern of `Box::from_raw(p: *mut T)`. Therefore, we mainly employ dataflow analysis to filter out incorrect options.

Algorithm 2 demonstrates the basic idea of our dataflow analysis method based on the AST of a function. Our method initially performs alias analysis via a simple union-find algorithm (lines 2–16). Then, we extract the potential identifiers with respect to a target pattern (line 17), for example, the parameters of the caller for the first pattern of unsafe constructors. Finally, we examine if our interested identifier (`unsafeParID`) of the target unsafe function usage is an alias of these potential identifiers (lines 18–22).

Algorithm 2 Pattern filtering via dataflow analysis

```

1: procedure PATTERNFILTER(func, pattern, unsafeParID) ▷ Given a function func with unsafe function calls, analyze if
   an unsafe function call with a parameter value unsafeParID meets the requirement of a particular pattern.
2:   for each assignExpr ∈ func do                                ▷ Extract alias sets by analyzing each assignment expression.
3:     (lval, rval) ← ExtractVal(assignExpr)
4:     if IsScalarType(lval) || IsScalarType(rval) then            ▷ Skip the case if either the left or right value is scalar type.
5:       continue;
6:     end if
7:     if !IsID(lval) then                                          ▷ Skip the case if the left value is not an identifier.
8:       continue;
9:     end if
10:    if IsID(rval) then
11:      Union(Find(lval), Find(rval))                                ▷ Apply the union find algorithm to the alias sets of the two identifiers.
12:    end if
13:    if IsFnCall(rval) then                                       ▷ If the right value is a function call.
14:      Union(Find(lval), Find(GetArgs(rval))) ▷ Establish alias relationships among the arguments and return value.
15:    end if
16:  end for
17:  ids ← GetPotentialIDs(func, pattern)                          ▷ E.g., function arguments, return values of function calls.
18:  for each id ∈ ids do
19:    if id ∈ Find(unsafeParID) then                                ▷ Find the alias set of unsafeParID established during alias analysis.
20:      return false;                                              ▷ The pattern should not be excluded.
21:    end if
22:  end for
23:  return true;                                                  ▷ The pattern should be excluded.
24: end procedure

```

4.3 | Model training

Our machine learning model consists of two steps. In the first step, we train a BERT [12] model, and the primary objective is to obtain a language model that learns the syntax and semantics of the Rust programming language from a substantial volume of Rust code. We chose BERT because it is a state-of-the-art architecture for learning from language or code data. In this way, the model can know the relationship between different program constructs. For example, converting a pointer type can either be achieved via `as *mut _` or `cast()`. We do not need to consider the variations of the same case in our knowledge base. The second step adds extra layers to fine-tuned BERT, and it aims to assess the similarity between two code snippets based on the code embeddings generated by the BERT model.

Figure 6a demonstrates the first step, which fine-tunes a BERT model with a substantial amount of Rust code from GitHub. The input to our model is the serialized AST generated in Section 4.1. During training, a masking mechanism is applied to randomly mask words in the input sequence. The model utilised in this step is an augmented version of the original BERT model, incorporating a linear layer and a softmax activation function. The linear layer is responsible for adjusting the dimensions of the BERT output to match the vocabulary length, while the softmax function produces the probability distribution of each word at every position in the sentence. To train the model, the original input AST traversal sequence is represented with a one-hot encoding representation. We employ the cross-entropy loss function to calculate the loss between the predicted and target probability distributions, facilitating training of the entire model. After enhanced training, we can obtain a 30×768 code embedding derived from code ASTs.

Figure 6b demonstrates the second step that trains an add-on neural network to align with our task. Note that the code embeddings generated by the BERT model only learn a common representation of Rust code. Since our task aims to learn the similarity of code snippets with respect to our summarized patterns, we should thereby train our model towards learning the differences between multiple patterns. To this end, we construct a two-layered, fully connected network and reduce the dimensionality of the code embeddings generated by BERT to 32. We employ the same neural network to compute embeddings of two code snippets and compare their similarity. If two code snippets belong to the same pattern, we label their similarity as 1; otherwise, we label their similarity as -1. In this training process, we employ cosine embedding loss and freeze the parameters of BERT.

5 | EVALUATION

In this section, we design experiments to evaluate whether our approach can accurately recommend the most similar unsafe code patterns for a given unsafe code snippet. Next, we discuss our detailed experimental setting and results.

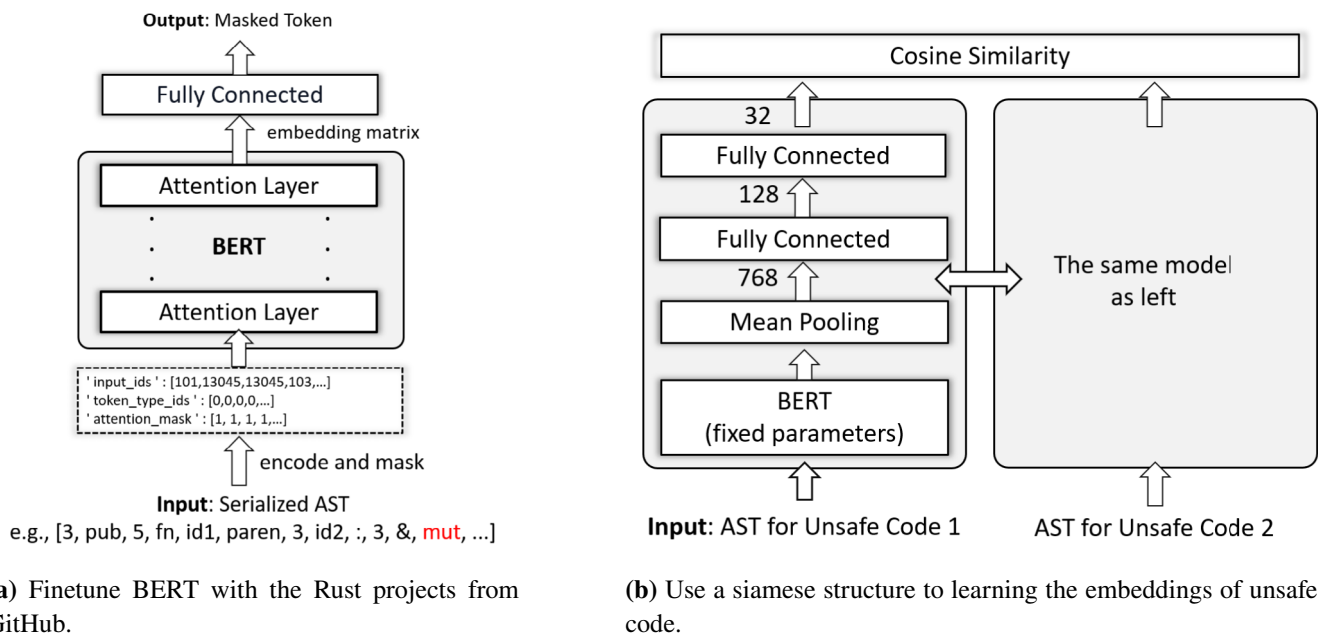


FIGURE 6 Machine learning approach to learn the embeddings of unsafe code.

5.1 | Experimental setting

5.1.1 | Prototype

We have developed a prototype system based on Rust 1.65.0 nightly, Rust syn crate 1.0.72 and Python 3.9.16. Our system consists of a Rust AST extractor and serializer, and a siamese neural network based on a pretrained BERT model. Our AST extractor utilises the Rust syn crate because it is powerful in generating AST for fragmented Rust code snippets. Our Siamese neural network is based on PyTorch 1.13.1.

Fine-tune BERT

To adapt the pretrained BERT model to Rust code, we fine-tune it with a large corpus of Rust code files downloaded from GitHub. The dataset includes 3442 Rust code files from 41 projects and 476 extra Rust code files with unsafe code to further strengthen the model for handling unsafe code. To comply with the input sequence length of the BERT model, we divided our serialized AST into code snippets of length 512. In total, we have extracted over 1×10^5 AST sequences to train the BERT model. We employ Adam optimizer [13] with a learning rate of 0.01 and train for 20 epochs.

Train siamese neural network

We train the fully connected layers of the neural network with unsafe code cases from our knowledge base and well-labelled GitHub samples. Our knowledge base currently contains 124 unsafe code snippets, and the GitHub dataset contains 472 samples. We randomly pair unsafe code from different patterns to generate training data with similarity -1 , and we pair unsafe code from the same pattern to generate training data with similarity 1 . To mitigate the data imbalance issues for some patterns with less data, we augment their unsafe code snippets by adding garbage code to generate more samples, such as introducing unused variable definitions and assertions. We also employ an Adam optimizer with a learning rate of 0.01 to train the model. We freeze the BERT layers and train the fully connected layers for 100 epochs.

5.1.2 | Dataset

Our dataset includes the data within the knowledge base and samples collected from GitHub. We employ k-fold cross-validation ($k = 3$) in our experiment. In each evaluation round, we employ two folds of GitHub samples as well as the knowledge base for training and employ the rest one-fold for evaluation.

Knowledge base

As stated in Section 3.2, we mainly use the unsafe code samples and their replacement suggestions from Rustdoc to build our knowledge base. However, the patterns in these code samples are not very rich. We further extend the knowledge base to cover all the patterns of unsafe APIs. Our resulting knowledge base contains 124 cases. Note that our knowledge base can be further enriched by considering more cases, and it will affect the recommendation accuracy and usability of the approach. We leave such investigations as our future work. In this work, we focus on validating the feasibility of a recommendation-based unsafe code mitigation idea.

GitHub data

To obtain enough data to train our model and evaluate its performance, we collect real-world unsafe code snippets for each unsafe API from the projects maintained on GitHub. Specifically, we first search the name of each unsafe API with the GitHub search engine and then locate the function that contains the unsafe API call as a candidate case. For each candidate, one student author manually labels its belonged unsafe pattern, and the other two student authors double check the correctness. We have finally collected 472 test cases. We will provide more detailed case information in Section 5.3.

5.1.3 | Baseline approach

We choose the following two existing approaches as our baseline.

- *SGNN* [8]: The work employs a siamese graph neural network to learn embeddings of unsafe code snippets and make recommendations. Different from this work, SGNN preprocesses unsafe code snippets into attributed control-flow

graphs for learning. The original work of SGNN employs 34 test cases of fixed GitHub issues in their study. In comparison, we have extended the dataset and removed all cases related to APIs with ‘unchecked’ in their identifiers.

- *GPT-3.5*: Large language models (LLM), as zero-shot learners [14] have achieved excellent performance in many tasks, and they have also been integrated into IDE tools to make code suggestions, for example, GitHub Copilot [15]. Therefore, we also examine the performance of LLM on our unsafe code replacement task. To our best knowledge, existing LLM tools do not support unsafe Rust code replacement directly. Therefore, we design our own prompt and employ ChatGPT as the baseline in our study. Our prompt template is shown as follows.

The following code is a Rust function that utilises an unsafe API, [API NAME]. Is there a safe alternative to this unsafe code? If so, please provide the corresponding safe code. You cannot change the signature of the current function. [CODE]

For each response of the GPT, we manually verify the result. We think GPT fails on the task if its response meets either of the three criteria: (1) incorrect answer of replaceability, that is, replaceable as replaceable, or vice versa; (2) the suggested code replacement introduces other unsafe APIs or has no security gain; and (3) the suggested code employs an inexistent API.

- *Tool by Zhang et al.* [16]: This tool supports nine patterns of replaceable unsafe code. However, four patterns are related to unchecked operations and one related to libc APIs. Therefore, we only evaluate the performance with the rest four patterns involving three APIs, `set_len`, `ptr::copy`, and `ptr::copy_nonoverlapping`. For fair comparison, we only choose replaceable code snippets within our dataset for testing.

5.2 | Results

Table 4 presents our experimental results. SafeNet can successfully recognize the patterns in 394 out of 472 cases. Besides, we also evaluated the results of the pure AST-based machine learning approach without using the filter, and the number of successful cases dropped to 247. Note that if one pattern has multiple cases, we employ the best average similarity to make the final decision. In comparison, SGNN is only successful with 129 cases, while GPT is successful with 147 cases. In addition, the tool by Zhang et al. [16] successfully recommended 13 out of only 51 cases.

Although SafeNet performs the best, it cannot achieve very high accuracy. On one hand, this is because the problem is very challenging; even the powerful GPT fails in most cases. On the other hand, our dataset is still very small, and there is still room to improve our approach and model training process.

5.3 | Case study

Now, we show the usefulness of our approach with several real-world cases from GitHub. Figure 7a is an example from the project `twitter/the-algorithm`. The code snippet creates a `Vec<T>` (line 4) and then employs `set_len` (line 6) to set the length of the vector before initializing its elements (line 8). Such usage of `set_len` may introduce vulnerable program points before the vector is fully initialized. For example, if the vector contains data of the `Drop` trait and the program panics when executing line 8, double free would occur. For such code, we recommend the PoC of Figure 7b and its alternative safer version (Figure 7c) to developers. Developers may either choose to initialize the vector with `vec!` when defining it or at least move `set_len` after the vector is fully initialized.

Figure 7d is another case with unsafe API `Vec::from_raw_parts`. The function converts an input of `Vec<T>` to `Vec<u8>`. The trick lies in the conversion of pointer types. We categorize it as pattern 5 (Figure 7e) and suggest an alternative safe method that reconstructs a new vector. However, developers may reject our suggestion and insist on using the unsafe version because it is obviously more efficient.

Figure 7g demonstrates a code snippet that shifts the content of a compound data structure, for example, array or slice. The code corresponds to the four patterns in our knowledge base. We thereby recommend the safe version of Figure 7h to developers. The safe API `copy_within` is also based on `ptr::copy` and incurs negligible overhead.

TABLE 4 Experimental result with unsafe APIs that have 3k+ usages on GitHub.

Unsafe API	Knowledge base			Result				
	# of patterns	# of cases	Test cases #	GPT	SGNN [8]	Zhang et al. [16]	SafeNet (no filter)	SafeNet
Box::from_raw	6	7	24	7	5	-	15	19
CStr::from_ptr	6	11	25	16	6	-	18	22
Vec::from_raw_parts	6	6	27	5	5	-	16	19
CString::from_raw	6	8	22	10	3	-	12	18
Rc::from_raw	6	9	20	8	1	-	13	16
mem::zeroed	2	4	12	5	8	-	10	12
assume_init	4	13	26	10	15	-	16	21
mem::transmute	6	11	30	15	7	-	17	19
mem::transmute_copy	6	10	22	8	8	-	16	17
set_len	3	3	34	11	9	9/22	20	20
ptr::copy_nonoverlapping	5	6	30	6	14	1/17	10	30
ptr::read	5	6	30	6	6	-	13	30
ptr::write	5	6	29	7	5	-	13	29
ptr::copy	5	6	31	4	9	3/12	14	31
ptr::write_bytes	5	6	27	9	6	-	11	27
offset_from	5	6	30	12	7	-	10	30
as_ref	4	4	20	5	6	-	9	20
* (raw_ptr deref)	4	4	33	3	9	-	14	14
Total	89	126	472	147	129	13/51	247	394

Note: For the tool by Zhang et al. [16], we only report the successful cases over supported cases.

Developers may reconsider if the safe function can meet their needs. In this way, we hope our approach can help developers mitigate the usage of unsafe code.

6 | RELATED WORK

Existing work on unsafe Rust mainly focuses on formal verification [17, 18], bug detection [5, 19] or isolation [20, 21]. Our work is novel from the perspective of unsafe code replacement.

There are two papers similar to this work, including SGNN [8] and another work by Zhang et al. [16]. SGNN is our previous conference paper with preliminary results presented. It employs graph neural networks with attributed control-flow graphs as the input. Different from SGNN, this work proposes a hybrid dataflow and AST-based approach for making recommendations and is more accurate. Moreover, this study is more in-depth with respect to the patterns of unsafe code usage and contains a larger dataset for evaluation. Besides, Zhang et al. [16] have conducted another concurrent work with respect to the necessity of unsafe code usage. They have summarized nine common patterns of unsafe code with safe alternatives and integrated the safe code suggestion feature into `rust-analyzer`. In comparison, our work considers all unsafe APIs, and we think each unsafe API could have multiple patterns. Our knowledge base should be more systematic and fine-grained. Besides, their pattern recognition method is ad-hoc and limited to these nine patterns only.

Our work employs machine learning models to learn code representations. There are many existing methods proposed in this direction, such as BERT [12], Code2Vec [22] and Code2Seq [23]. BERT can directly learn from code text, and it utilises a multi-layer transformer architecture and contains a large number of parameters. After extensive training on data with word masking, the BERT model itself captures a substantial amount of code semantics and syntax information. Therefore, BERT can be directly used for the task at hand in this context. However, BERT may be influenced by non-code information, so the key challenge lies in extracting the Rust semantic information contained within BERT. Additionally, if Rust code is directly inputted, BERT may be affected by variable names and other interfering information, leading to suboptimal results. Hence, we chose to incorporate a neural network model to align with our specific task. Code2Vec [22] and Code2Seq [23] focus on learning from different paths in the code AST. These models incorporate attention mechanisms to extract AST features and have achieved good results. We were inspired by learning

```

1 pub fn tensor_to_vec <T: kind::Element>(res: &Tensor)
2 -> Vec<T> {
3     let size = TorchModel::tensor_flatten_size(res);
4     let mut res_f32: Vec<T> = Vec::with_capacity(size);
5     unsafe {
6         res_f32.set_len(size);
7     }
8     res.copy_data(res_f32.as_mut_slice(), size);
9     res_f32
10 }

```

(a) set_len : sample code from project: twitter/the-algorithm.

```

1 fn foo(vec: &mut Vec<Vec<u8>>) { .. }
2 //fn foo(vec: *mut Vec<u8>) { ... }
3 fn main() {
4     let mut v: Vec<Vec<u8>> = vec! [Vec::new(); 10];
5     //panic here safe;
6     foo( &mut v);
7     //foo(v.as_mut_ptr());
8     //unsafe { v.set_len(10); } //should move it after foo
9 }

```

(c) set_len : safe case of pattern 2 our knowledge base.

```

1 #![feature(vec_into_raw_parts)]
2 fn convert (v: Vec<u8>) -> Vec<i8> {
3     let (p, l, c) = v.into_raw_parts();
4     unsafe { Vec::from_raw_parts(p as *mut i8, l, c) }
5 }

```

(e) Vec::from_raw_parts : unsafe case of pattern 2 in our knowledge base.

```

1 pub fn shift (&mut self) {
2     if self .position > 0 { unsafe {
3         let length = self .end - self .position;
4         ptr::copy(
5             self .memory[ self .position .. self .end].as_ptr(),
6             self .memory[ .. length].as_mut_ptr(), length,);
7         self .position = 0;
8         self .end = length;
9     }}

```

(g) ptr::copy : sample code from project: sozu-proxy/sozu.

```

1 fn foo(vec: &mut Vec<Vec<u8>>) { .. }
2 }
3 //fn foo(vec: *mut Vec<u8>) { ... }
4
5 fn main() {
6     let mut v: Vec<Vec<u8>> = Vec::with_capacity( 10);
7     unsafe { v.set_len( 10); }
8     //panic here is unsafe;
9     foo( &mut v);
10    //foo(v.as_mut_ptr());
11 }

```

(b) set_len : unsafe case of pattern 2 in our knowledge base.

```

1 pub fn cast_vec <T: Copy>( mut vec: Vec<T>) -> Vec<u8> {
2     let len = std::mem::size_of:: <T>() * vec.len();
3     let cap = std::mem::size_of:: <T>() * vec.capacity();
4     let ptr = vec.as_mut_ptr();
5     std::mem::forget(vec);
6     unsafe { Vec::from_raw_parts(ptr as _, len, cap) }
7 }
8

```

(d) Vec::from_raw_parts : sample code from project: amethyst/rendy.

```

1 fn convert (v: Vec<u8>) -> Vec<i8> {
2     let mut output: Vec<i8> = Vec::with_capacity(v.len());
3     for byte in v { output.push(byte as i8); }
4     output
5 }

```

(f) Vec::from_raw_parts : safe case of pattern 2 in our knowledge base.

```

1 fn copy_last_half_to_beginning (slice: &mut [u8]) {
2     let len = slice.len();
3     let half_len = len / 2;
4     // Create a temporary vector
5     let mut v = Vec::with_capacity(half_len);
6     v.extend_from_slice( &slice[half_len .. ]);
7     slice.copy_within( .. half_len, half_len);
8     slice[ .. half_len].copy_from_slice( &temp_vec);
9 }

```

(h) ptr::copy : safe case of pattern 4 in our knowledge base.

FIGURE 7 Case study with real-world unsafe code samples from GitHub.

embeddings from the AST level. However, since our task targets the tokens of the AST leaf nodes and is independent of the AST path, we do not choose Code2Vec but instead fine-tune BERT for our task.

7 | CONCLUSIONS

To summarize, this paper has studied the problem of how to help Rust developers mitigate the usage of unsafe code. We have systematically studied the usage patterns of unsafe code that can be replaced with safe code. We found some unsafe codes are misused because they can be replaced with negligible cost, while other unsafe code replacements may

incur substantial overhead. Therefore, we modelled the problem of unsafe code mitigation as a recommendation problem and let developers decide whether the replacement cost is acceptable. Based on the idea, we have built a knowledge base for popular unsafe APIs in the Rust standard library. Moreover, we have proposed a hybrid approach that employs dataflow analysis to rule out incorrect usage patterns and leverages machine learning to suggest similar cases in our knowledge base. Experimental results with real-world unsafe code snippets show that our approach can achieve a promising recommendation accuracy, and the suggested cases are useful.

DATA AVAILABILITY STATEMENT

Our source code and dataset are available online: <https://github.com/Artisan-Lab/SafeNet>.

REFERENCES

- Shin J, Kwon D, Seo J, Cho Y, Paek Y. CRCCount: Pointer invalidation with reference counting to mitigate use-after-free in legacy C/C++. *Network and Distributed System Security (NDSS) Symposium* 2019.
- Monteiro FR, Gadelha MR, Cordeiro LC. Model checking C++ programs. *Softw Test Verif Reliab (STVR)*. 2022;32(1):e1793.
- Tsai PA, Gan YL, Sanchez D. Rethinking the memory hierarchy for modern languages. *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)* 2018: 203–16.
- Astrauskas V, Matheja C, Poli F, Müller P, Summers AJ. How do programmers use unsafe rust? *Proc ACM Prog Lang (PACMPL)*. 2020; 4(OOPSLA):1–27.
- Bae Y, Kim Y, Askar A, Lim J, Kim T, Rudra: Finding memory safety bugs in Rust at the ecosystem scale. 2021: 84–99.
- Fulton KR, Chan A, Votipka D, Hicks M, Mazurek ML. Benefits and drawbacks of adopting a secure programming language: Rust as a case study. *Seventeenth Symposium on Usable Privacy and Security (SOUPS)* 2021: 597–616.
- Höllerer S, Klostermeyer P, Wöhler N, Acar Y, Fahl S. Poster: Unsafe Rust – conscious choice or spiky shortcut? *In 43rd IEEE Symposium on Security and Privacy (SOSP)* 2022.
- Lu X, Zhang Z, Xu H. Safe Rust code recommendation based on Siamese Graph Neural Network. *Pacific Rim International Symposium on Dependable Computing (PRDC)* 2022: 1–11.
- Evans AN, Campbell B, Soffa ML. Is Rust used safely by software developers? *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE)* 2020: 246–57.
- Rice HG. Classes of recursively enumerable sets and their decision problems. *Trans Am Math Soc*. 1953;74(2):358–66. <https://doi.org/10.1090/S0002-9947-1953-0053041-6>
- Rust Syn library. <https://docs.rs/syn/latest/syn/>; 2023.
- Devlin J, Chang MW, Lee K, Toutanova K. BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* 2018.
- Kingma DP, Ba J. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* 2014.
- Wei J, Bosma M, Zhao VY, et al. Finetuned language models are Zero-Shot learners. *10th International Conference on Learning Representations (ICLR)* 2022.
- Nguyen NT, Nadi S. An empirical evaluation of GitHub Copilot's code suggestions. *2022 IEEE/ACM 19th International Conference on Mining Software Repositories (MSR)* 2022: 1–5.
- Zhang Y, Kundu A, Portokalidis G, Xu J. On the dual nature of necessity in use of Rust unsafe code. *Proceedings of the 31th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)* 2023.
- Jung R, Jourdan JH, Krebbers R, Dreyer D. RustBelt: securing the foundations of the rust programming language. *Proc ACM Program Lang (POPL)*. 2017;2(POPL):1–34. <https://doi.org/10.1145/3158154>
- Matsushita Y, Denis X, Jourdan JH, Dreyer D. A semantic foundation for functional verification of Rust programs with unsafe code. *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)* 2022.
- Cui M, Chen C, Xu H, Zhou Y. SafeDrop: detecting memory deallocation bugs of rust programs via static data-flow analysis. *ACM Trans Softw Eng Methodol (TOSEM)*. 2022;32(4):1–21. <https://doi.org/10.1145/3542948>
- Liu P, Zhao G, Huang J. Securing unsafe Rust programs with XRust. *2020 IEEE/ACM 42nd International Conference on Software Engineering (ICSE)* 2020: 234–45.
- Rivera E, Mergendahl S, Shrobe H, Okhravi H, Burrow N. Keeping safe Rust safe with Galeed. *Annual Computer Security Applications Conference (ACSAC)* 2021.
- Alon U, Zilberstein M, Levy O, Yahav E. Code2vec: learning distributed representations of code. *Proc ACM Program Lang (POPL)*. 2019;3(POPL):1–29. <https://doi.org/10.1145/3290353>
- Alon U, Brody S, Levy O, Yahav E. Code2seq: Generating sequences from structured representations of code. *ArXiv* 2018; abs/1808.01400.

How to cite this article: Dong Y, Zhang Z, Cui M, Xu H. SafeNet: Towards mitigating replaceable unsafe Rust code via a recommendation-based approach. *Softw Test Verif Reliab*. 2024;34(5):e1875. <https://doi.org/10.1002/stvr.1875>